Extension Plan: Architecture improvements

Edouard Fousson 327774
Thomas Picart 355524
Jonathan Balli 373029
COM-304 Extension Plan

***Abstract*—This project explores architectural improvements to a small-scale Transformer model (nano4M). We investigate modifications to core architectural components in order to improve training stability, optimization behavior, and final performance. The proposed changes are evaluated through controlled ablation studies against a nano4M baseline model.**

I. Introduction

This project belongs to the **Architecture Improvements** category defined in Section 4 of the project guidelines, which focuses on modifying internal Transformer components while keeping the training setup and learning objective unchanged.

Starting from a **nano4M baseline Transformer**, we explore five architectural modifications targeting its main building blocks: positional encoding (RoPE and alternatives), the feed-forward network (SwiGLU), weight initialization strategies, model depth, and residual path design. Even at small scale, these components strongly influence training stability and final performance — standard choices such as sinusoidal encodings, ReLU MLPs, and unscaled residuals can lead to suboptimal convergence or limited expressivity. Each modification is evaluated through controlled ablation studies, isolating its individual effect on training dynamics and validation performance.

II. Related Works

The Transformer architecture was introduced in [1], where self-attention replaced recurrence and became the foundation for modern sequence models. While highly effective, the original formulation leaves several design choices open, particularly in positional encoding, feed-forward structure, and optimization stability.

A key improvement direction concerns positional representations. Rotary Positional Embeddings (RoPE) [2] improve the modeling of relative positions by encoding them directly into attention computations, leading to better extrapolation and more stable sequence modeling. Similarly, ALiBi-style bias methods [3] demonstrate that simple linear biases can also replace explicit positional embeddings while preserving performance.

Feed-forward network design has also evolved beyond standard ReLU-based MLPs. Gated activations such as SwiGLU [4] improve gradient flow and expressivity by introducing multiplicative gating, and are widely used in modern Transformer variants due to their empirical gains.

Training stability is strongly influenced by normalization and residual design. Pre-LayerNorm Transformers and scaling strategies such as DeepNorm-inspired approaches [8] help stabilize deep networks by improving gradient propagation through residual connections.

Finally, attention efficiency has been improved through variants such as Multi-Query Attention (MQA) [9], which shares key and value projections across heads to reduce redundancy while maintaining competitive performance.

Overall, prior work shows that many gains in Transformer performance come from small but well-targeted architectural refinements rather than large structural changes. This motivates our systematic evaluation of these improvements on a nano4M baseline model.

III. Method

Our approach consists of five targeted architectural modifications to the nano4M baseline Transformer, each addressing a distinct structural component. All modifications are designed to be independently applicable, enabling controlled ablation studies. The training objective and dataset remain unchanged across all variants.

A. Data

All model variants are trained on the pre-tokenized multimodal dataset provided as part of the nano4M exercise. This dataset comprises aligned modality pairs including RGB images, depth maps, surface normals, semantic segmentation maps, and text captions, encoded into discrete token sequences using pre-provided modality-specific tokenizers. No additional data collection or preprocessing is required for this extension.

B. Architectural Modifications

We implement five modular modifications to the nano4M baseline, each targeting a distinct component. Positional encodings are replaced by RoPE [2] or ALiBi [3] to improve relative position modeling. The feed-forward block is replaced by a SwiGLU variant [4] for improved expressivity. Weight initialization is revisited using He [5], Xavier [6], or DeepNorm-inspired [8] strategies to stabilize gradient propagation. Model depth is varied to study capacity-compute trade-offs. Finally, residual scaling is introduced to control update magnitudes and improve training stability [8]. Full implementation details and equations are provided in Appendix A.

C. Training Procedure

All variants are trained from scratch using identical hyperparameters to the nano4M baseline (optimizer, learning rate schedule, batch size, and total number of training tokens). Only

TABLE I
SUMMARY OF ARCHITECTURAL MODIFICATIONS.

Modification	Component	Key change
Positional embeddings	Attention	RoPE [2] / ALiBi [3]
Feed-forward block	MLP	SwiGLU [4]
Weight initialization	All layers	He [5] / Xavier [6] / DeepNorm [8]
Depth variation	Global	$\pm$layers vs. baseline
Residual scaling	Residual path	$\alpha \cdot F(x)$ + Pre-LN [8]

the target architectural component differs between runs. Validation loss is logged at regular intervals throughout training to capture both convergence speed and final performance.

IV. VALIDATION

A. Metrics

Our primary metric is the **per-token validation cross-entropy loss**, evaluated on a held-out split of the multimodal dataset. This metric is directly comparable across all variants under the same training budget. As a secondary metric, and consistent with the project guidelines, we report the **Fréchet Inception Distance (FID)** [7] on generated RGB images, which measures the perceptual quality and diversity of image outputs relative to the real data distribution. We additionally monitor **gradient norm** trajectories throughout training as a proxy for optimization stability, which is particularly informative for modifications targeting residual connections and initialization.

To support performance-efficiency trade-off analysis in the final report, we also record the following hardware metrics for each variant under identical conditions:

- training throughput (tokens per second);
- peak GPU memory consumption;
- wall-clock time to reach predefined validation loss thresholds;
- parameter count and estimated FLOPs per forward pass.

B. Experiments and Ablations

We adopt a controlled ablation protocol in which each modification is evaluated independently against the unmodified nano4M baseline, with all other components held fixed. This isolates the contribution of individual design choices. Concretely, the ablation study includes the following runs:

- *Baseline*: unmodified nano4M Transformer
- *Positional embeddings*: RoPE vs. ALiBi vs. baseline
- *Feed-forward*: SwiGLU vs. baseline ReLU MLP
- *Initialization*: He / Xavier / DeepNorm-style vs. default
- *Depth*: shallow variant / deep variant vs. baseline
- *Residual scaling*: scaled residuals vs. unscaled baseline
- *Combined*: best-performing modifications stacked together

The combined variant is included to assess whether individual gains compose additively or exhibit interactions. All runs use identical training budgets (same number of tokens seen), batch sizes, and learning rate schedules.

C. Baselines

The primary baseline is the **unmodified nano4M Transformer** trained under the same conditions as all experimental variants. Within each modification family, self-baselines are used to distinguish between specific design choices (e.g., RoPE vs. ALiBi, He vs. Xavier initialization). No external pretrained models are used as baselines, as the focus is on isolating the effect of architectural changes within a controlled training setup.

V. TIMELINE

TABLE II
WEEK-BY-WEEK PLAN AND ESTIMATED COMPUTE REQUIREMENTS.

Week	Tasks	Compute
1	Implement RoPE + alternative positional encoding (Alibi), SwiGLU, weight initialization; run baseline sanity checks and short debug runs per variant.	Low (short sanity runs)
2	Implement depth variation and residual scaling; launch controlled ablation runs; track validation loss, stability, and compute cost.	Moderate (parallel ablations)
3	Select most promising variants; launch final training runs; compare against baseline; analyze learning curves; prepare figures and result tables.	High (definitive runs)
4	Write final report; finalize tables and visualizations; organize reproducibility details and submission material.	Minimal

The most **compute-intensive** period is Week 3, which concentrates the definitive training runs for the most promising architectural variants. Weeks 1 and 4 remain light, and Week 2 requires moderate compute distributed across parallel ablation runs. All runs share the same hardware setup to ensure comparability of reported efficiency metrics.

VI. DISCUSSION

- **Risks and limitations.** The main risk is that some architectural changes may not produce clear gains within the available training budget. In particular, improved initialization and residual scaling may mostly improve training stability and early optimization, while having only limited impact on final validation loss. Another practical limitation is that the different modifications may interact: improvements measured in isolated ablations may not transfer directly once several changes are combined in the final model. Finally, some variants increase implementation and benchmarking cost, which reduces the amount of time available for clean comparisons.
- **Possible improvements with more time/compute.** With more time and compute, we would extend this study to stronger attention-level changes, such as gated attention, local attention, grouped-query attention, or a broader exploration of MQA. We would also run larger sweeps over depth, width, and initialization strategies, and test whether the best-performing modifications remain beneficial at larger scale and longer training budgets.

APPENDIX A
ARCHITECTURAL MODIFICATION DETAILS

Positional Embeddings. The nano4M baseline uses learned absolute positional embeddings. We replace these with Rotary Positional Embeddings (RoPE) [2], which encode relative position information by rotating query and key vectors in each attention head, improving relative position modeling and sequence length generalization. As a second candidate, we evaluate ALiBi [3], which introduces a linear bias on attention logits without explicit positional parameters, offering a parameter-free alternative.

SwiGLU Feed-Forward Network. The baseline feed-forward sublayer applies a two-layer MLP with ReLU activation. We replace it with a SwiGLU block [4]:

$$\text{FFN}_{\text{SwiGLU}}(x) = \big(\text{SiLU}(xW_1) \otimes xW_2\big)\, W_3 \tag{1}$$

where $\otimes$ denotes element-wise multiplication and $\text{SiLU}(z) = z \cdot \sigma(z)$. This gating mechanism improves gradient flow and representational expressivity.

Weight Initialization. We evaluate He initialization [5], which scales weights by $\sqrt{2/n_{\text{in}}}$ and targets non-linear activations such as SiLU, and Xavier initialization [6], which scales by $\sqrt{2/(n_{\text{in}} + n_{\text{out}})}$ for variance preservation. We additionally consider a DeepNorm-inspired rescaling [8], multiplying residual branch weights by $\beta = (8N)^{-1/4}$ where N is the number of Transformer layers.

Depth Variation. We train a shallower and a deeper variant relative to the nano4M baseline, keeping all other hyperparameters constant, to characterize the depth–performance trade-off under a fixed training budget.

Residual Path Modifications. Each residual branch output is scaled by:

$$x \leftarrow x + \alpha \cdot F(x), \quad \alpha = \frac{1}{\sqrt{N}} \tag{2}$$

where N is the number of Transformer layers. Inspired by DeepNorm [8], this controls the magnitude of residual updates and stabilizes training in deeper configurations. Pre-Layer Normalization is verified in the baseline and applied if absent.

REFERENCES

[1] A. Vaswani et al., "Attention Is All You Need," NeurIPS, 2017.
[2] J. Su et al., "RoFormer: Enhanced Transformer with Rotary Position Embedding," arXiv:2104.09864, 2021.
[3] O. Press et al., "Train Short, Test Long: Attention with Linear Biases (ALiBi)," arXiv:2108.12409, 2021.
[4] N. Shazeer, "GLU Variants Improve Transformer," arXiv:2002.05202, 2020.
[5] K. He, X. Zhang, S. Ren, and J. Sun, "Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification," ICCV, 2015.
[6] X. Glorot and Y. Bengio, "Understanding the Difficulty of Training Deep Feedforward Neural Networks," AISTATS, 2010.
[7] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, "GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium," NeurIPS, 2017.
[8] G. Wang et al., "DeepNet: Scaling Transformers to 1,000 Layers," arXiv:2203.00555, 2022.
[9] N. Shazeer, "Fast Transformer Decoding: One Write-Head is All You Need," arXiv:1911.02150, 2019.